



Univerzitet u Novom Sadu
Fakultet tehničkih nauka



Dokumentacija za projektni zadatak

Studenti: Marija Mitrašinović, SV42/2024

Jovan Savić, SV14/2024

Nikola Anđelković, SV38/2024

Nikola Pešić, SV39/2024

Predmet: Nelinearno programiranje i evolutivni algoritmi

Broj projektnog zadatka: 13

Tema projektnog zadatka: Veštačke Neuronske mreže

Opis problema

Cilj ovog projektnog zadatka bio je implementacija veštačke neuronske mreže potpuno od nule, bez korišćenja gotovih biblioteka za mašinsko učenje. Neuronska mreža treba da omogućiti:

- Proizvoljan broj slojeva i neurona po sloju
- Izbor različitih aktivacionih funkcija
- Treniranje pomoću različitih gradijentnih metoda optimizacije

Neuronska mreža se koristi za fitovanje krivih, odnosno za aproksimaciju nepoznate funkcije na osnovu skupa ulazno-izlaznih podataka.

Na primer, ako imamo podatke tipa (x, y) gde je $y = x^2$, mreža treba da nauči ovu zavisnost i da za nove ulazne vrednosti x može da predvidi odgovarajuće izlazne vrednosti y .

Matematički, problem možemo formulisati ovako:

- Dato: Skup treniranih podataka $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Cilj: Naučiti funkciju f takvu da $f(x) \approx y$

Naša neuronska mreža se sastoji od:

1. Ulaznog sloja - prima ulazne podatke
2. Skrivenih slojeva - obrađuju podatke i ekstraktuju karakteristike
3. Izlaznog sloja - daje konačnu predikciju

Svaki sloj se sastoji od neurona koji su povezani sa svim neuronima iz prethodnog sloja. Svaka veza ima svoju težinu (weight), a svaki neuron ima svoj pomeraj (bias).

Uvod

Veštačka neuronska mreža je matematički model inspirisan biološkim neuronskim mrežama. Sastoji se od međusobno povezanih jedinica zvanih neuroni (ili čvorovi), organizovanih u slojeve.

Osnovni princip rada i arhitektura:

1. Neuron prima više ulaza.
2. Svaki ulaz se množi sa odgovarajućom težinom.
3. Sabiraju se svi ponderi i dodaje se pomeraj (bias).
4. Rezultat prolazi kroz aktivacionu funkciju.
5. Izlaz se prosleđuje neuronima u sledećem sloju.

Jedan neuron računa sledeće:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b \quad a = \text{sigma}(z)$$

Gde su:

$x_1, x_2, \dots, x_n$ - ulazi u neuron
 $w_1, w_2, \dots, w_n$ - težine (weights) svake veze
 b - pomeraj (bias)
 z - ponderisana suma (weighted sum)
 sigma - aktivaciona funkcija
 a - izlaz neurona (aktivacija)

Aktivacione funkcije unose nelinearnost u mrežu, što joj omogućava da aproksimira složene funkcije. Implementirali smo sledeće aktivacione funkcije:

1. Sigmoid

$$\text{sigma}(x) = 1 / (1 + e^{(-x)})$$
$$\text{sigma}'(x) = \text{sigma}(x) \cdot (1 - \text{sigma}(x))$$

- + Vrednosti između 0 i 1
- + Dobra za binarnu klasifikaciju

2. Tanh (Hiperbolički tangens)

$$\sigma(x) = \tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

$$\sigma'(x) = 1 - \tanh^2(x)$$

+ Vrednosti između -1 i 1

+ Često bolja od sigmoid za skrivene slojeve

3. ReLU (Rectified Linear Unit)

$$\sigma(x) = \max(0, x)$$

$$\sigma'(x) = 1 \text{ ako } x > 0, \text{ inače } 0$$

+ Najčešće korišćena u modernim mrežama

+ Brza za računanje

4. Leaky ReLU

$$\sigma(x) = x \text{ ako } x > 0, \text{ inače } \alpha \cdot x$$

$$\sigma'(x) = 1 \text{ ako } x > 0, \text{ inače } \alpha$$

- Varijanta ReLU koja rešava problem "umirućih neurona"

5. Linear (Linearna) - Koristi se u izlaznom sloju za regresiju

$$\sigma(x) = x$$

$$\sigma'(x) = 1$$

Za merenje greške koristimo Mean Squared Error (MSE):

$$MSE = (1/N) \cdot \sum (y_i - \hat{y}_i)^2$$

Gde je:

y_i - stvarna vrednost

$\hat{y}_i$ - predviđena vrednost

N - broj primera

Cilj treniranja je minimizovanje ove greške.

Implementacija

Neuroni i Neuronska mreža

1. Klasa Neuron

Klasa Neuron predstavlja osnovnu jedinicu neuronske mreže. Svaki neuron sadrži:

- Vektor težina (weights) - težine svih veza sa neuronima iz prethodnog sloja
- Pomeraj (bias) - dodatni parametar koji se dodaje na ponderisanu sumu
- Izlaz (output) - rezultat nakon primene aktivacione funkcije
- Delta vrednost - koristi se tokom backpropagation algoritma za računanje gradijenata

Struktura klase Neuron

```
cpp

class Neuron {

private:

    std::vector<double> weights; // Težine veza sa prethodnim slojem

    double bias; // Pomeraj neurona

    double output; // Izlaz nakon aktivacije

    double delta; // Delta vrednost za backpropagation

public:

    Neuron(int numWeights); ~Neuron() {}

    double activate(const std::vector<double> &inputs, const ActivationFunc
&activationFunction);

    void initializeWeights(int numWeights);

    // Getteri i setteri

    double getOutput() const { return output; }

    void setOutput(double val) { output = val; }

    const std::vector<double> &getWeights() const { return weights; }
```

```

void setWeights(const std::vector<double> &vals) { weights = vals; }

double getBias() const { return bias; }

void setBias(double val) { bias = val; }

double getDelta() const { return delta; }

void setDelta(double val) { delta = val; }

};

```

Inicijalizacija težina

Prilikom kreiranja neurona, težine se inicijalizuju nasumičnim vrednostima iz uniformne distribucije u intervalu [-1, 1].

Ovaj pristup omogućava razbijanje simetrije i bolji početak procesa učenja.

```

cpp
void Neuron::initializeWeights(int numWeights) {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(-1.0, 1.0);
    weights.resize(numWeights);
    for (int i = 0; i < numWeights; ++i)
    {
        weights[i] = dis(gen);
    }
    bias = dis(gen);
}

```

Aktivacija neurona

Metoda activate izračunava izlaz neurona prema sledećoj formuli:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b \quad a = \sigma(z)$$

Gde je:

- $x_1, x_2, \dots, x_n$ - ulazne vrednosti
- $w_1, w_2, \dots, w_n$ - težine odgovarajućih veza
- b - pomerač (bias) z - ponderisana suma (net input)
- σ - aktivaciona funkcija a - izlaz neurona (aktivacija)

cpp

```
double Neuron::activate(const std::vector<double> &inputs, const ActivationFunc &func)
{
    if (inputs.size() != weights.size()) {
        throw std::invalid_argument("Input size does not match weights size.");
    }

    double netInput = bias; // Počni sa bias vrednošću

    for (size_t i = 0; i < inputs.size(); ++i)
    {
        netInput += weights[i] * inputs[i]; // Ponderisana suma
    }

    output = func(netInput); // Primeni aktivacionu funkciju

    return output;
}
```

Objašnjenje koraka:

1. Proveri da li je broj ulaza jednak broju težina
2. Inicijalizuj ponderisanu sumu sa vrednošću bias-a
3. Za svaki ulaz, pomnoži ga sa odgovarajućom težinom i dodaj na sumu
4. Primeni aktivacionu funkciju na rezultat
5. Sačuvaj i vrati izlaz

Klasa Layer

Klasa Layer predstavlja sloj neurona u mreži. Svaki sloj se sastoji od proizvoljnog broja neurona koji su potpuno povezani sa prethodnim slojem (fully connected layer).

Struktura klase Layer

cpp

```
class Layer {  
  
private:  
  
    std::vector<Neuron> neurons; // Vektor neurona u sloju  
  
public:  
  
    Layer(int numNeurons, int numInputsPerNeuron); ~Layer() {}  
  
    std::vector<double> forward(const std::vector<double> &inputs,  
                                const ActivationFunc &activationFunction);  
  
    std::vector<double> getOutputs() const;  
  
    // Getteri  
  
    std::vector<Neuron> &getNeurons() { return neurons; }  
  
    const std::vector<Neuron> &getNeurons() const { return neurons; }  
  
    int size() const { return neurons.size(); }  
  
};
```

Konstruktor sloja

Konstruktor kreira zadati broj neurona, gde svaki neuron ima onoliko težina koliko ima neurona u prethodnom sloju (ili ulaza za prvi skriveni sloj).

cpp

```
Layer::Layer(int numNeurons, int numInputsPerNeuron) {  
  
    neurons.reserve(numNeurons); // Rezerviši memoriju za efikasnost  
  
    for (int i = 0; i < numNeurons; ++i) {
```



```

        neurons.emplace_back(numInputsPerNeuron); // Kreiraj neuron sa datim
        brojem ulaza
    }
}

```

Forward pass kroz sloj

Metoda forward prosleđuje ulazne podatke kroz sve neurone u sloju i vraća vektor izlaza.

cpp

```

std::vector<double> Layer::forward(const std::vector<double> &inputs, const
ActivationFunc &func) {

    std::vector<double> outputs;

    outputs.reserve(neurons.size());

    for (Neuron &neuron : neurons) {

        outputs.push_back(neuron.activate(inputs, func)); // Aktiviraj svaki neuron
    }

    return outputs;
}

```

Dobijanje izlaza sloja

cpp

```

std::vector<double> Layer::getOutputs() const {

    std::vector<double> outputs;

    outputs.reserve(neurons.size());

    for (const Neuron &neuron : neurons) {

        outputs.push_back(neuron.getOutput());
    }

    return outputs;
}

```

```
}
```

Klasa Network

Klasa Network predstavlja kompletnu neuronsku mrežu. Omogućava:

- Proizvoljan broj slojeva i neurona po sloju
- Izbor aktivacione funkcije za celu mrežu
- Izbor optimizacione metode (Steepest Descent, ADAGRAD, RMSProp, ADAM)
- Treniranje i predikciju

Struktura klase Network

```
cpp
```

```
class Network {
```

```
private:
```

```
    std::vector<Layer> layers; // Slojevi mreže
```

```
    std::vector<int> layerSizes; // Broj neurona u svakom sloju
```

```
    Activation activation; // Aktivaciona funkcija
```

```
    std::unique_ptr<Optimizer> optimizer; // Optimizator
```

```
    // Skladištenje gradijenata
```

```
    std::vector<std::vector<std::vector<double>>> weight_gradients; //  
[sloj][neuron][težina]
```

```
    std::vector<std::vector<double>> bias_gradients;           // [sloj][neuron]
```

```
public:
```

```
    // Konstruktor
```

```
    Network(const std::vector<int>& layerSizes, const Activation& activation,  
            OptimizerType optimizerType, double learningRate);
```

```
    // Konfiguracija
```

```
    void setOptimizer(OptimizerType optimizerType, double learningRate);
```

```

void setActivation(const Activation& newActivation);

void initializeGradients();

std::unique_ptr<Optimizer> createOptimizer(OptimizerType type, double
learningRate);


std::vector<double> forward(const std::vector<double>& inputs);

void backward(const std::vector<double>& inputs, const std::vector<double>&
targets);

void train(const std::vector<double>& inputs, const std::vector<double>&
targets);

void fit(const std::vector<std::vector<double>>& inputData,

        const std::vector<std::vector<double>>& targetData,

        int epochs, bool verbose = false);

std::vector<double> predict(const std::vector<double>& inputs);

double calculateLoss(const std::vector<std::vector<double>>& inputData,

                    const std::vector<std::vector<double>>& targetData);

};

```

Konstruktor mreže

Konstruktor prima:

- layerSizes - vektor koji definiše broj neurona u svakom sloju (npr. {2, 3, 1} za 2 ulaza, 3 skrivena neurona, 1 izlaz)
- activation - aktivaciona funkcija koja se koristi u celoj mreži
- optimizerType - tip optimizatora za ažuriranje težina
- learningRate - brzina učenja

cpp

```

Network::Network(const std::vector<int>& layerSizes, const Activation& activation,
OptimizerType optimizerType, double learningRate) : layerSizes(layerSizes),
activation(activation) {

    // Kreiraj slojeve

    // Prvi indeks (layerSizes[0]) predstavlja broj ulaza, ne sloj

    for (size_t i = 1; i < layerSizes.size(); ++i) {

        layers.emplace_back(layerSizes[i], layerSizes[i - 1]);

    }

    // Inicijalizuj skladište za gradijente

    initializeGradients();

    // Kreiraj optimizator

    optimizer = createOptimizer(optimizerType, learningRate);

}

```

Objašnjenje:

Ako je layerSizes = {2, 3, 1}:

layerSizes[0] = 2 - broj ulaza (nije sloj, već broj ulaznih vrednosti)

layerSizes[1] = 3 - prvi skriveni sloj sa 3 neurona, svaki ima 2 težine

layerSizes[2] = 1 - izlazni sloj sa 1 neuronom, koji ima 3 težine

Inicijalizacija gradijenata

Gradijenti se čuvaju u trodimenzionalnoj strukturi za težine i dvodimenzionalnoj za bias-e:

```

cpp
void Network::initializeGradients() {
    weight_gradients.resize(layers.size());
    bias_gradients.resize(layers.size());
    for (size_t l = 0; l < layers.size(); ++l) {

```

```
int numNeurons = layerSizes[l + 1]; // Broj neurona u trenutnom sloju
int numInputs = layerSizes[l];      // Broj ulaza (neurona iz prethodnog sloja)

weight_gradients[l].resize(
    numNeurons,
    std::vector<double>(numInputs, 0.0)
);

bias_gradients[l].resize(numNeurons, 0.0);
}
}
```

Forward Propagation

Forward propagation (propagacija unapred) je proces prosleđivanja ulaznih podataka kroz mrežu, sloj po sloj, do izlaznog sloja. Svaki sloj prima izlaze prethodnog sloja kao svoje ulaze.

Algoritam

1. Postavi ulazne podatke kao početne vrednosti
2. Za svaki sloj u mreži: a. Prosleđuj trenutne ulaze kroz sve neurone sloja b. Svaki neuron izračunava ponderisanu sumu i primenjuje aktivacionu funkciju c. Izlazi neurona postaju ulazi za sledeći sloj
3. Vрати izlaze poslednjeg (izlaznog) sloja

Matematička formulacija

Za sloj l sa n neurona:

$$\text{Ulaz u neuron } j: z_j^{(l)} = \sum (w_{ji}^{(l)} \cdot a_i^{(l-1)}) + b_j^{(l)}$$

$$\text{Izlaz neurona } j: a_j^{(l)} = \sigma(z_j^{(l)})$$

Gde je:

- $a_i^{(l-1)}$ - izlaz neurona i iz prethodnog sloja
- $w_{ji}^{(l)}$ - težina veze između neurona i iz prethodnog sloja i neurona j iz trenutnog sloja
- $b_j^{(l)}$ - pomeraj (bias) neurona j u sloju l
- σ - aktivaciona funkcija

Implementacija

```
cpp
std::vector<double> Network::forward(const std::vector<double>& inputs) {
    std::vector<double> currentInputs = inputs; // Početni ulazi
    for (auto& layer : layers) {
        // Prosleđuj kroz svaki sloj, izlazi postaju novi ulazi
        currentInputs = layer.forward(currentInputs, activation.func);
    }

    return currentInputs; // Vрати izlaze poslednjeg sloja
}
```

Objašnjenje koraka:

1. Inicijalizuj currentInputs sa ulaznim podacima

2. Za svaki sloj pozovi `layer.forward()`
3. Metoda `forward` sloja aktivira sve neurone i vraća njihove izlaze
4. Izlazi sloja postaju ulazi za sledeći sloj
5. Nakon prolaska kroz sve slojeve, vrati konačne izlaze

Prednosti implementacije:

- Modularnost - Svaki sloj nezavisno obrađuje ulaze
- Fleksibilnost - Lako dodavanje novih slojeva
- Čitljivost - Jasan tok podataka kroz mrežu

Back Propagation

Back propagation predstavlja osnovni algoritam za treniranje višeslojnih veštačkih neuronskih mreža. Njegova uloga je da omogući efikasno računanje gradijenata greške u odnosu na sve parametre mreže (težine i bias-e), kako bi se oni mogli ažurirati pomoću odgovarajuće gradijentne metode optimizacije.

Algoritam se sastoji od dva glavna koraka:

- forward propagation - izračunavanje izlaza mreže,
- backward propagation - propagacija greške unazad i računanje gradijenata.

U ovom radu back propagation je implementiran ručno, bez korišćenja biblioteka za automatsko diferenciranje, čime je omogućena potpuna kontrola nad procesom učenja.

Osnovna ideja

Cilj back propagation algoritma je minimizacija funkcije greške (u našem slučaju Mean Squared Error - MSE). To se postiže računanjem parcijalnih izvoda greške po svakom parametru mreže i njihovim korišćenjem za korekciju vrednosti tih parametara.

Osnovni princip je primena lančanog pravila izvoda (chain rule), koje omogućava da se greška sa izlaza mreže prenese unazad, sloj po sloj, sve do ulaznog sloja.

Algoritam

1. Forward pass

Prvo se izvršava forward propagation:

- ulazni podaci se prosleđuju kroz sve slojeve mreže,
- za svaki neuron se izračunava ponderisana suma i primenjuje aktivaciona funkcija,
- izlazi svih slojeva se pamte jer su potrebni u backward fazi.

2. Izlazni sloj - računanje greške i delta vrednosti

Za izlazni sloj greška se računa direktno poređenjem predviđene i stvarne vrednosti.

Za svaki neuron izlaznog sloja:

- Greška:
$$\text{error} = \text{output} - \text{target}$$
- Delta vrednost:
$$\delta = \text{error} \cdot \sigma'(\text{output})$$

gde je:

- *output* - izlaz neurona,
- *target* - očekivana vrednost,
- σ' - izvod aktivacione funkcije.

Delta vrednost predstavlja meru uticaja datog neurona na ukupnu grešku.

3. Skriveni slojevi - propagacija greške unazad

Za skrivene slojeve greška se ne može direktno izračunati, već se dobija kao zbir uticaja neurona iz sledećeg sloja.

Za svaki neuron u skrivenom sloju:

- Izračunavanje greške:
$$\text{error} = \sum(\text{weight_next} \cdot \text{delta_next})$$
- Izračunavanje delta vrednosti:
$$\delta = \text{error} \cdot \sigma'(\text{output})$$

Na ovaj način greška se postepeno prenosi od izlaznog sloja ka ulaznom, uzimajući u obzir jačinu veza (težine) između neurona.

4. Računanje gradijenata

Nakon što su delta vrednosti izračunate za sve neurone, moguće je izračunati gradijente funkcije greške u odnosu na parametre mreže.

Za svaki neuron i svaki sloj važi:

- Gradijent po bias-u:
$$\partial L / \partial b = \delta$$
- Gradijent po težini:
$$\partial L / \partial w = \delta \cdot \text{input}$$

gde je *input* izlaz neurona iz prethodnog sloja (ili ulaz mreže za prvi skriveni sloj).

Izračunati gradijenti se ne koriste direktno za ažuriranje težina, već se prosleđuju odgovarajućoj gradijentnoj metodi optimizacije.

Veza sa gradijentnim metodama

Back propagation algoritam ne vrši ažuriranje težina samostalno. Njegov zadatak je isključivo da obezbedi tačne gradijente.

Na osnovu tih gradijenata, parametri mreže se ažuriraju pomoću jedne od implementiranih optimizacionih metoda:

- Steepest Descent
- ADAGRAD
- RMSProp
- ADAM

Ovakvo razdvajanje odgovornosti omogućava modularnost implementacije i lako poređenje različitih optimizatora uz isti back propagation algoritam.

Prednosti implementiranog pristupa

- Potpuna kontrola nad procesom učenja
- Jasno razdvajanje:
 - forward propagation
 - back propagation
 - optimizacije parametara
- Laka proširivost (dodavanje novih optimizatora ili funkcija greške)
- Edukativna vrednost - algoritam je implementiran od osnovnih matematičkih principa

Gradijentne metode optimizacije

Nakon što izračunamo gradijente, potrebno je ažurirati težine i biase. Postoji više načina kako to da uradimo

0. Hijerarhija klasa (Optimizer)

Sve metode nasleđuju apstraktnu klasu `Optimizer` koja ima promenjivu *learning_rate* (brzina učenja) i čistu virtuelnu funkciju *update* koju svaka klasa naslednica (klasa koja predstavlja gradijentnu metodu) implementira.

```
cpp
class Optimizer {
protected:
    double learning_rate; // Brzina učenja

public:
    virtual void update(
        std::vector<std::vector<double>>& weights,
        std::vector<double>& biases,
        const std::vector<std::vector<double>>& weight_gradients,
        const std::vector<double>& bias_gradients
    ) = 0; // Čista virtuelna funkcija
};
```

1. Metod najbržeg pada (Steepest Descent)

Najjednostavnija metoda optimizacije. Ideja je da se parametri ažuriraju tako što se pomeraju u suprotnom pravcu od gradijenta.

Način rada:

1. Izračunaj gradijente za sve težine i biase (urađeno u backpropagation)
2. Za svaku težinu: oduzmi gradijent pomnožen brzinom učenja
3. Za svaki bias: oduzmi gradijent pomnožen brzinom učenja
4. Ponovi za sve slojeve

Matematički:

$$w_new = w_old - \alpha * gradient_w$$
$$b_new = b_old - \alpha * gradient_b$$

Gde je: alpha brzina učenja (learning rate).
w (weight) je težina veze
b (bias) je pomeraj

Ključni deo koda:

```
// Za svaki neuron u sloju
for (size_t i = 0; i < num_neurons; i++) {
    // Ažuriranje svih težina
    for (size_t j = 0; j < num_inputs; j++) {
        weights[i][j] -= learning_rate * weight_gradients[i][j];
    }

    // Ažuriranje biasa
    biases[i] -= learning_rate * bias_gradients[i];
}
```

Karakteristike:

(+ označava pozitivne, - označava negativne karakteristike)

- + Jednostavna implementacija
- + Nema dodatnu memoriju
- Ista brzina učenja za sve parametre
- Može biti spora
- Osetljiva na izbor brzine učenja

Tipična brzina učenja: 0.01 - 0.1

2. ADAGRAD (Adaptive Gradient)

ADAGRAD automatski prilagođava brzinu učenja za svaki parametar zasebno, na osnovu istorije njegovih gradijenata. Parametri sa većim gradijentima dobijaju manje ažuriranje, a oni sa manjim gradijentima dobijaju veće ažuriranje.

Način rada:

1. Inicijalizuj cache (keš memoriju) na nule za sve parametre
2. Izračunaj gradijente (urađeno u backpropagation)
3. Za svaki parametar: dodaj kvadrat trenutnog gradijenta u njegov cache
4. Izračunaj prilagođenu brzinu učenja: $\alpha / \sqrt{\text{cache} + \epsilon}$
5. Ažuriraj parametar sa prilagođenom brzinom učenja
6. Ponovi za sve težine i biase

*Cache se nikad ne resetuje - akumulira se tokom celog treniranja

Matematički:

```
cache = cache + (gradient)^2  
prilagodjena_brzina = alpha / sqrt(cache + epsilon)  
w_new = w_old - prilagodjena_brzina * gradient
```

Gde je:

cache - akumulirana suma kvadrata gradijenata
epsilon - mali broj (10^{-8}) za numeričku stabilnost

Ključni deo koda:

```
// Akumuliranje kvadrata gradijenta u cache  
weight_cache[i][j] += weight_gradients[i][j] * weight_gradients[i][j];  
  
// Ažuriranje sa prilagođenom brzinom učenja  
weights[i][j] -= (learning_rate / std::sqrt(weight_cache[i][j] + epsilon)) * weight_gradients[i][j];
```

Zašto je cache bitan:

Cache čuva informaciju o tome koliko se parametar menjao u prošlosti. Ako je cache veliki (parametar se često značajno menjao), prilagođena brzina učenja postaje manja. Ako je cache mali, brzina učenja ostaje veća.

Karakteristike:

- + Automatski prilagođava brzinu učenja
- + Dobro radi sa retkim podacima
- Brzina učenja kontinualno opada
- Može prerano zaustaviti učenje

3. RMSProp (Root Mean Square Propagation)

RMSProp je poboljšanje ADAGRAD-a. Koristi pokretni prosek (moving average) kvadrata gradijenata umesto njihove akumulacije, što sprečava da brzina učenja preterano opadne.

Kako radi (korak po korak):

1. Inicijalizuj cache na nule
2. Izračunaj gradijente (urađeno u backpropagation)
3. Za svaki parametar: ažuriraj cache pomoću pokretnog proseka
Zadrži deo starog cache-a $(\text{decay_rate} * \text{stari_cache})$
Dodaj deo novog gradijenta $((1 - \text{decay_rate}) * \text{gradient}^2)$
4. Izračunaj prilagođenu brzinu učenja: $\alpha / \sqrt{\text{cache} + \text{epsilon}}$
5. Ažuriraj parametar
6. Ponovi za sve težine i biase

Matematički:

```
cache = ro * cache + (1 - ro) * (gradient)^2
```

```
prilagodjena_brzina = alpha / sqrt(cache + epsilon)
w_new = w_old - prilagodjena_brzina * gradient
```

Gde je:

ro - faktor smanjenja (decay rate), obično 0.9 ili 0.99

Ključni deo koda:

```
// Pokretni prosek kvadrata gradijenata
weight_cache[i][j] = decay_rate * weight_cache[i][j] +
    (1.0 - decay_rate) * weight_gradients[i][j] * weight_gradients[i][j];

// Ažuriranje sa prilagođenom brzinom
weights[i][j] -= (learning_rate / std::sqrt(weight_cache[i][j] + epsilon)) * weight_gradients[i][j];
```

Razlika od ADAGRAD-a: ADAGRAD sabira sve kvadrate gradijenata (cache raste stalno), dok RMSProp koristi pokretni prosek (stari gradijenti postepeno "zaboravljaju se"). To omogućava mreži da nastavi da uči i nakon mnogo iteracija.

Karakteristike:

- + Ne pati od problema opadajuće brzine učenja
- + Dobro radi u praksi
- + Stabilniji od ADAGRAD-a
- Dodatni hiperparametar (decay_rate)

Tipične vrednosti:

Brzina učenja: 0.001 - 0.01
Decay rate: 0.9 ili 0.99

4. ADAM (Adaptive Moment Estimation)

ADAM je najsofisticiraniji optimizier koji kombinuje prednosti momentum metode i RMSProp-a. Održava dva pokretna proseka: prosek gradijenata (prvi moment) i prosek kvadrata gradijenata (drugi moment).

Kako radi (korak po korak):

1. Inicijalizuj m (prvi moment) i v (drugi moment) na nule
2. Inicijalizuj timestep (t) na 0
3. Izračunaj gradijente (urađeno u backpropagation)
4. Uvećaj timestep: $t = t + 1$
5. Ažuriraj m (prosek gradijenata):
 - o $m = \beta_1 * m + (1 - \beta_1) * \text{gradient}$
 - o Ovo daje "inerciju" - pamtimo pravac kretanja
6. Ažuriraj v (prosek kvadrata gradijenata):
 - o $v = \beta_2 * v + (1 - \beta_2) * \text{gradient}^2$

- Ovo prilagođava brzinu učenja kao u RMSProp
- 7. Koriguj m zbog početne pristrasnosti (bias correction):
 - $m_korigovano = m / (1 - \beta_1^t)$
- 8. Koriguj v zbog početne pristrasnosti:
 - $v_korigovano = v / (1 - \beta_2^t)$
- 9. Izračunaj korak ažuriranja:
 - $korak = (\alpha * m_korigovano) / (\sqrt{v_korigovano} + \epsilon)$
- 10. Ažuriraj parametar: $w_new = w_old - korak$

Matematički:

$m = \beta_1 * m + (1 - \beta_1) * gradient$ (prvi moment)
 $v = \beta_2 * v + (1 - \beta_2) * gradient^2$ (drugi moment)

$m_korigovano = m / (1 - \beta_1^t)$ (korekcija)
 $v_korigovano = v / (1 - \beta_2^t)$ (korekcija)

$w_new = w_old - (\alpha * m_korigovano) / (\sqrt{v_korigovano} + \epsilon)$

Gde je:

m - prvi moment (momentum - prosek gradijenata)
 v - drugi moment (prosek kvadrata gradijenata)
 β_1, β_2 - faktori smanjenja (obično 0.9 i 0.999)
 t - broj iteracija (timestep)

Ključni delovi koda:

```

    // Uvećaj timestep
    timestep++;

    // Faktori za korekciju pomeraja
    double bias_correction1 = 1.0 - std::pow(beta1, timestep);
    double bias_correction2 = 1.0 - std::pow(beta2, timestep);

    ...

    // Ažuriranje prvog momenta (momentum)
    weight_m[i][j] = beta1 * weight_m[i][j] +
        (1.0 - beta1) * weight_gradients[i][j];

    // Ažuriranje drugog momenta (RMSProp-like)
    weight_v[i][j] = beta2 * weight_v[i][j] +
        (1.0 - beta2) * weight_gradients[i][j] * weight_gradients[i][j];

    ...

    // Korigovani momenti
    double m_korigovano = weight_m[i][j] / bias_correction1;
    double v_korigovano = weight_v[i][j] / bias_correction2;
  
```

```
// Konačno ažuriranje
weights[i][j] -= (learning_rate * m_korigovano) /
                 (std::sqrt(v_korigovano) + epsilon);
```

Zašto korekcija pomeraja:

Na početku treniranja (male vrednosti t), m i v su blizu nule jer su inicijalizovani na nulu. Korekcija pomeraja kompenzuje ovu početnu pristrasnost i omogućava optimizeru da efikasno radi od samog početka.

Karakteristike:

- + Kombinuje prednosti momentum i RMSProp metoda
- + Obično najbolje radi u praksi
- + Uključuje korekciju pristrasnosti za početne iteracije
- + Manje osetljiv na izbor hiperparametara
- Više memorije (čuva m i v za svaki parametar)
- Malo sporiji po iteraciji od jednostavnijih metoda

Tipične vrednosti:

Brzina učenja (alpha): 0.001 - 0.01

Beta1: 0.9

Beta2: 0.999

Epsilon: 10^{-8}

Eksperimenti i testiranje

Validacija na XOR problemu:

U cilju provere ispravnosti implementacije neuronske mreže izvršeno je testiranje na XOR problemu. XOR predstavlja nelinearno separabilan problem, te zahteva najmanje jedan skriveni sloj, što ga čini pogodnim za validaciju višeslojne mreže.

Osnovna arhitektura korišćena u eksperimentima bila je: {2, 3, 1}

MSE (Mean Squared Error) korišćen je kao mera performansi.

Uticaj optimizatora

U ovom eksperimentu analiziran je uticaj različitih optimizacionih metoda, dok su svi ostali parametri ostali nepromenjeni.

Optimizator	Finalni MSE	Brzina konvergencije
Steepest Descent	≈ 0.00057	Srednja
ADAGRAD	≈ 0.00070	Brza u početku
RMSProp	≈ 0.00071	Brza
ADAM	≈ 0.00060	Najbrža

Iz tabele se može zaključiti da adaptivne metode (RMSProp i ADAM) postižu bržu i stabilniju konvergenciju u odnosu na klasični metod najbržeg pada.

Uticaj learning rate parametra

Testirane su vrednosti learning rate parametra: 0.5, 0.1, 0.01 i 0.001.

Learning Rate	Finalni MSE	Uočeno ponašanje
0.5	≈ 0.00071	Brza konvergencija
0.1	≈ 0.00064	Stabilna konvergencija

0.01	≈ 0.00073	Sporije učenje
0.001	≈ 0.00072	Najsporija konvergencija

Rezultati pokazuju da čak i mali broj neurona može uspešno rešiti XOR problem. Povećanje broja neurona ubrzava učenje, ali ne donosi značajno poboljšanje konačne greške za jednostavan zadatak.

Uticaj broja skrivenih slojeva

Testirane su sledeće konfiguracije:

{2,3,1}, {2,3,3,1}, {2,5,5,1}

Arhitektura	Finalni MSE	Zaključak
{2,3,1}	≈ 0.00060	Dovoljno
{2,3,3,1}	≈ 0.00064	Bez značajnog poboljšanja
{2,5,5,1}	≈ 0.00064	Veća kompleksnost

Zaključak:

Jedan skriveni sloj je dovoljan za rešavanje XOR problema. Povećanje dubine povećava kompleksnost bez značajnog poboljšanja performansi.

Uticaj broja neurona u skrivenom sloju

Analizirane su sledeće arhitekture:

{2,2,1}, {2,3,1}, {2,5,1}, {2,10,1}

Arhitektura	Finalni MSE	Zaključak
-------------	-------------	-----------

$\{2,2,1\}$	≈ 0.00046	Dovoljno za XOR
$\{2,3,1\}$	≈ 0.00089	Stabilno
$\{2,5,1\}$	≈ 0.00071	Brža konvergencija
$\{2,10,1\}$	≈ 0.00085	Bez značajnog poboljšanja

Rezultati pokazuju da čak i mali broj neurona može uspešno rešiti XOR problem. Povećanje broja neurona ubrzava učenje, ali ne donosi značajno poboljšanje konačne greške za jednostavan zadatak.

Testiranje na regresionom problemu

Pored klasifikacionog problema, mreža je testirana i na zadatku aproksimacije funkcije $\sin(x)$. U ovom slučaju korišćena je tanh aktivaciona funkcija, pogodna za funkcije definisane u intervalu $[-1,1]$.

Rezultati su pokazali da mreža uspešno aproksimira sinusnu funkciju, uz opadanje MSE tokom treninga, čime je potvrđena sposobnost modela da rešava i regresione zadatke.

Zaključak

U okviru ovog projektnog zadatka uspešno je implementirana višeslojna veštačka neuronska mreža od nule, bez korišćenja postojećih biblioteka za mašinsko učenje. Realizovani su svi ključni elementi sistema: forward propagation, back propagation algoritam, više aktivacionih funkcija i različite gradijentne metode optimizacije.

Poseban značaj imala je ručna implementacija back propagation algoritma, čime je omogućeno detaljno razumevanje načina na koji se greška propagira kroz mrežu i kako se računaju gradijenti parametara. Modularna organizacija koda, kroz hijerarhiju optimizatora, omogućila je jednostavno poređenje metoda kao što su Steepest Descent, ADAGRAD, RMSProp i ADAM.

Eksperimentalni rezultati na XOR problemu pokazali su da mreža uspešno rešava nelinearno separabilne zadatke, dok je testiranje na regresionom problemu aproksimacije funkcije $\sin(x)$ potvrdilo njenu sposobnost da rešava i kontinuirane probleme. Analiza uticaja brzine učenja, broja slojeva i neurona dodatno je pokazala kako arhitektura i hiperparametri utiču na brzinu i stabilnost konvergencije.

Može se zaključiti da je cilj projekta u potpunosti ostvaren: razvijena je funkcionalna i proširiva implementacija neuronske mreže koja jasno demonstrira principe nelinearne optimizacije i učenja na osnovu podataka.

**Napomene su iz primera dokumentacije*

Napomena 1: Poglavlja data u ovom šablonu su predlog koncepta dokumentacije. Ukoliko želite, možete ih menjati ili dodavati nova poglavlja.

Napomena 2: Nemojte kopirati ceo programski kod u dokumentaciju. Dozvoljeno je kopiranje kraćih delova koda u dokumentaciju ukoliko pomaže prilikom diskusije nekog aspekta problema/rešenja.